



ru slide (note)

Chapter 1

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/ba123944-ae40-4e8d-bea1-ffcc0c06bffb/Chapter-1_Introduction_and_Overview.pdf

Data Structure #revise-1

Data structure is a way of organizing and storing data in a computer in order to efficiently perform operations on that data. It provides a systematic way to organize and manage data, enabling quick access, retrieval, and manipulation.

Why Data Structure? OR Need Of Data Structure #revise-1

1. **Efficient Data Organization:** Data structures provide efficient ways to organize and store data. By choosing the appropriate data structure, we can optimize the use of memory and access data quickly.
2. **Data Manipulation:** Data structures offer various operations to manipulate data, such as insertion, deletion, searching, sorting, and traversal. These operations are fundamental for solving complex problems efficiently and effectively.
3. **Problem Solving:** Many real-world problems can be represented and solved more easily by using appropriate data structures. For example, a tree data structure is suitable for representing hierarchical relationships, and a graph data structure is useful for modeling relationships between objects.
4. **Code Readability and Maintainability:** Using appropriate data structures makes code more readable and maintainable. When data is organized and accessed

in a structured manner, it becomes easier for other developers to understand and maintain the codebase.

Linear Data Structure #revise-1

A linear data structure is a type of data structure where data elements are organized sequentially or linearly. It means that each element has a predecessor and a successor, except for the first and last elements.

Arrays are useful for storing and accessing data elements with constant-time access, while linked lists are efficient for dynamic data structures with easy insertions and deletions.

Some common types of linear data structures:

1. **Arrays:** Arrays are a basic form of a linear data structure, where elements are stored in contiguous memory locations.
2. **Linked Lists:** Linked lists consist of nodes where each node holds a data element and a reference to the next node.
3. **Stacks:** Stacks follow the Last-In-First-Out (LIFO) principle, where elements are inserted and removed from one end called the top.
4. **Queues:** Queues follow the First-In-First-Out (FIFO) principle, where elements are inserted at one end called the rear and removed from the other end called the front.
5. **Deque (Double-Ended Queue):** Deques are similar to queues but allow insertion and removal of elements from both ends.

Non-Linear Data Structure #revise-1

A non-linear data structure is a type of data structure where data elements are not organized sequentially or linearly.

Some common types of non-linear data structures:

1. **Trees:** Trees are hierarchical structures composed of nodes, where each node can have zero or more child nodes. Nodes in a tree are connected by edges, and they have a parent-child relationship.

2. Graphs: Graphs consist of a set of vertices (or nodes) and edges that connect these vertices. Unlike trees, graphs can have arbitrary connections between nodes, allowing for more complex relationships.
3. Heaps: Heaps are specialized trees that satisfy the heap property, which defines the ordering of elements in the tree. The most common type of heap is the binary heap.

Array #revise-1

It is a collection of elements of the same data type, stored in contiguous memory locations. The elements in an array are accessed using their indices, which indicate their position in the array.

Some key features and characteristics of arrays:

1. Fixed Size: Arrays have a fixed size, which is determined at the time of declaration. Once an array is created, its size cannot be changed dynamically.
2. Indexing: Elements in an array are accessed using their indices. The index of the first element is usually 0, and the index of the last element is the size of the array minus one.
3. Homogeneous Elements: Arrays store elements of the same data type. For example, an array of integers will only contain integer values, and an array of strings will only contain string values.
4. Memory Overhead: Arrays have a fixed memory overhead, meaning that even if the array is not fully utilized, memory is allocated for the entire declared size.

Some common use cases include:

- Storing and manipulating collections of data, such as lists of numbers, names, or coordinates.
- Implementing matrices, tables, and multi-dimensional data structures.
- Efficiently accessing elements in search algorithms like binary search.
- Supporting other data structures, such as stacks and queues, which can be implemented using arrays.

Linked List #revise-1

A linked list is a data structure that consists of a sequence of nodes, where each node contains data and a reference (or link) to the next node in the sequence. Linked lists provide a flexible way to store and manipulate data dynamically.

Some key features and characteristics of linked lists:

1. **Node Structure:** Each node in a linked list contains two components: data and a reference (or pointer) to the next node. The data component holds the actual value, and the reference component points to the next node in the sequence.
2. **Dynamic Size:** Linked lists can grow or shrink dynamically during program execution. Unlike arrays, which have a fixed size, linked lists can easily accommodate new nodes or remove existing nodes.
3. **No Contiguous Memory Allocation:** Unlike arrays, linked lists do not require contiguous memory allocation. Each node can be located anywhere in memory.
4. **Insertion and Deletion:** Linked lists provide efficient insertion and deletion operations. Adding or removing a node involves updating the references between nodes.
5. **Sequential Access:** Linked lists support sequential access, where traversal starts from the first node and continues until the last node. To access a specific node, one needs to traverse the list from the beginning until reaching the desired node.
6. **Singly Linked List and Doubly Linked List:** In a singly linked list, each node has a reference only to the next node in the sequence. In a doubly linked list, each node has references to both the next node and the previous node, allowing for backward traversal.
7. **Circular Linked List:** In a circular linked list, the last node of the list points back to the first node, creating a circular structure.

Linked lists have various applications and use cases, such as:

- Implementing dynamic data structures like stacks and queues.

- Efficiently handling insertions and deletions in scenarios where the size of the data may change.
- Building hash tables, graph representations, and other complex data structures.
- Implementing memory management techniques in programming languages.

Advantages & Disadvantages of Linked List

Advantage	Disadvantage
Linked lists can grow or shrink as needed by adding or removing nodes.	Elements in a linked list cannot be accessed directly by index. To access, we need to traverse the list from the beginning.
Adding or removing nodes in a linked list can be done efficiently.	Linked lists require additional memory to store the link pointers
Linked lists can easily utilize dynamic memory allocation.	Accessing an element in a linked list takes linear time
Linked lists are relatively easy to implement and manipulate.	Linked lists are not suitable for scenarios where random access to elements is required.

Stack

A stack is a fundamental data structure that follows the Last-In-First-Out (LIFO) principle. It represents a collection of elements in which new elements are added or removed from the top of the stack.

Some key features and characteristics of a stack:

1. **LIFO Order:** The last element added to the stack is the first one to be removed. Elements are inserted and removed from only one end, commonly referred to as the top of the stack.
2. **Operations:** Stacks typically support two primary operations:
 - **Push:** Adds an element to the top of the stack.
 - **Pop:** Removes and returns the topmost element from the stack.
3. **Peeking:** The peek operation allows to examine the top element of the stack without removing it.

4. Implementation: Stacks can be implemented using arrays or linked lists. Array-based stacks have a fixed size, while linked list-based stacks can dynamically grow or shrink as needed.

Stacks find applications in various scenarios, such as:

- Evaluating arithmetic expressions using postfix notation.
- Balancing parentheses and checking syntax validity.
- Depth-First Search (DFS) traversal in graphs.
- Tracking function calls and managing local variables.
- Undo/redo functionality in text editors and design tools.
- Backtracking algorithms and recursive solutions.

Queue

A queue is a fundamental data structure that follows the First-In-First-Out (FIFO) principle. It represents a collection of elements in which new elements are added at one end called the rear (enqueue) and removed from the other end called the front (dequeue).

Some key features and characteristics of a queue:

1. FIFO Order: The first element added to the queue is the first one to be removed. Elements are added to the rear and removed from the front.
2. Operations: Queues typically support two primary operations:
 - Enqueue: Adds an element to the rear of the queue.
 - Dequeue: Removes and returns the element from the front of the queue.
3. Peek: The peek operation allows you to examine the element at the front of the queue without removing it.
4. Implementation: Queues can be implemented using arrays or linked lists. Array-based queues have a fixed size, while linked list-based queues can dynamically grow or shrink as needed.
5. Circular Queue: Circular queues are a variation of queues where the rear and front "wrap around" to the beginning of the underlying data structure, creating

a circular arrangement.

6. Applications: Queues find applications in various scenarios, such as:

- Managing shared resources in a multi-threaded or concurrent environment.
- Implementing breadth-first search (BFS) traversal in graphs.
- Handling print spooling and job scheduling.
- Simulating real-world scenarios like call centers or waiting lines.

Tree

A tree is a widely used data structure that represents a hierarchical structure composed of nodes. It is a collection of elements called nodes, where each node contains data and has zero or more child nodes.

Some key features and characteristics of trees:

1. Node Structure: Each node in a tree consists of a data component and zero or more references (or links) to its child nodes.
2. Root: The topmost node of a tree is called the root. It serves as the starting point for accessing and navigating the entire tree.
3. Parent and Child Nodes: Nodes in a tree have a hierarchical relationship. A node that has child nodes is called a parent node, and the nodes directly below a parent node are its child nodes.
4. Siblings: Nodes that share the same parent are called siblings. They are at the same level in the tree hierarchy.
5. Leaf Nodes: Leaf nodes are nodes that do not have any child nodes. They are the terminal nodes in the tree.
6. Depth and Height: The depth of a node in a tree is the number of edges from the root to that node. The height of a tree is the maximum depth of any node in the tree.
7. Binary Trees: Binary trees are a special type of tree where each node has at most two child nodes, commonly referred to as the left child and the right child.

8. **Balanced Trees:** Balanced trees are binary trees that ensure the height difference between the left and right subtrees is minimized, improving the efficiency of search, insertion, and deletion operations.
9. **Tree Traversal:** Tree traversal refers to the process of visiting and accessing nodes in a tree in a specific order.
10. **Applications:** Trees are used in various applications, such as:
 - Organizing hierarchical data, like file systems or organization structures.
 - Implementing search and indexing structures, such as binary search trees.
 - Parsing and representing expressions, like abstract syntax trees (ASTs).
 - Representing and analyzing data relationships, like decision trees in machine learning.
 - Efficiently storing and retrieving data, like B-trees in databases.

Graph

A graph is a widely used data structure that represents a collection of interconnected nodes or vertices. It is composed of a set of vertices and a set of edges, where each edge connects two vertices and represents a relationship or connection between them.

Some key features and characteristics of graphs:

1. **Vertices:** Vertices (also called nodes) are the fundamental units of a graph. Each vertex represents an object or entity, and it can contain additional information or properties.
2. **Edges:** Edges represent the connections or relationships between vertices. They can be directed or undirected, depending on whether the relationship has a specific direction or not.
3. **Adjacency:** If there is an edge between two vertices, they are said to be adjacent or neighbors.
4. **Weighted Edges:** Graph edges can be assigned weights or costs, which represent additional information about the relationship between the vertices.

5. Degree: The degree of a vertex is the number of edges connected to it. In a directed graph, the degree can be further divided into in-degree (incoming edges) and out-degree (outgoing edges) of a vertex.
6. Directed and Undirected Graphs: In a directed graph (also called a digraph), edges have a specific direction from one vertex to another. In an undirected graph, edges have no specific direction, and the relationship is bidirectional.
7. Cycle: A cycle is a path in a graph that starts and ends at the same vertex, allowing for traversal through multiple edges.
8. Connectedness: In a connected graph, there is a path between any two vertices. In a disconnected graph, there is no path between vertices in different components.
9. Applications: Graphs find applications in various domains, such as:
 - Social networks, representing relationships between individuals.
 - Routing and network analysis, modeling connections between routers or nodes.
 - Web page ranking algorithms, like Google's PageRank.
 - Recommendation systems, suggesting items based on similarity or connections.
 - Transportation networks, optimizing routes and logistics.
 - Dependency analysis in software engineering and project management.

Chapter 4

Operations Performed On Any Linear Structure

1. Traversal: Traversal involves accessing each element in the linear structure one by one. It allows us to process or examine each element individually. Traversal can be done from the beginning to the end (forward traversal) or from the end to the beginning (backward traversal). For arrays, we can use a loop to iterate over each element based on its index. ~~For linked lists, you start from the head or starting point and follow the links until the end of the list.~~

2. Search: The search operation is used to find a specific element within the linear structure. It determines whether the element exists in the structure and, if so, returns its position or index.
3. Insertion: Insertion involves adding a new element to the linear structure. Depending on the position where the element is to be inserted, the structure may need to be adjusted to accommodate the new element. For linked lists, we can create a new node, adjust the references, and insert it at the appropriate location.
4. Deletion: Deletion involves removing an element from the linear structure. Similar to insertion, deletion may require adjustments to maintain the integrity of the structure. In arrays, you can delete an element by shifting the remaining elements to fill the gap left by the deleted element.
5. Sorting: Sorting refers to arranging the elements of a linear structure in a specific order, such as ascending or descending.
6. Merging: Merging is the process of combining two or more linear structures into a single structure while maintaining a specific order. This operation is typically used when working with sorted structures. For example, you can merge two sorted arrays into a single sorted array or merge two sorted linked lists into a single sorted linked list.

Linear Array

A linear array, also known as a one-dimensional array, is a basic data structure that stores a collection of elements in a linear sequence. It provides a contiguous block of memory to store elements of the same data type. Each element in the array is accessed by its index, which represents its position in the sequence.

Total Number of Elements: To calculate the total number of elements in a linear array, you can use the formula:

$$\text{Total Elements} = (\text{Last Index} - \text{First Index}) + 1$$

Representation of Linear Array in Memory

Study through youtube

Traversing a Linear Array #revise-1

Traversing a linear array means visiting and accessing each element of the array in a specific order. It allows you to process or examine each element individually. There are generally two common ways to traverse a linear array: using a loop or recursion.

1. Loop-Based Traversal:

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int length = sizeof(arr) / sizeof(arr[0]);

    // Loop-based traversal
    for (int i = 0; i < length; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

2. Recursion-based Traversal:

```
#include <stdio.h>

void traverseArray(int arr[], int index, int length) {
    // Base case: Stop recursion when index exceeds the length
    if (index >= length) {
        return;
    }

    // Accessing the current element at the given index
    printf("%d ", arr[index]);
}
```

```

        // Recursive call with the next index
        traverseArray(arr, index + 1, length);
    }

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int length = sizeof(arr) / sizeof(arr[0]);

    // Recursion-based traversal
    traverseArray(arr, 0, length);

    return 0;
}

```

Inserting an Element #revise-1

Inserting an element into a linear data structure, such as an array or a linked list, involves adding a new element at a specific position within the existing elements.

General steps for inserting an element at a specific index in an array:

- Determine the position where you want to insert the new element, such as at the beginning, middle, or end of the array.
- Shift all the elements to the right of the insertion position to create an empty space for the new element.
- Insert the new element at the desired position.
- Adjust the length of the array to reflect the addition of the new element.

Inserting an element at a specific index in an array:

Algorithm:

1. Initialize the array with existing elements.
2. Set the length of the array.
3. Set the index where the element is to be inserted.
4. Set the new element to be inserted.

5. Shift elements to the right, starting from the last element and moving towards the desired index.
6. Insert the new element at the specified index.
7. Increase the length of the array by 1.
8. Print the elements of the array.

```
#include <stdio.h>

int main() {
    int arr[10] = {10, 20, 30, 40, 50};
    int length = 5;
    int index = 2; // Index at which the element is to be inserted
    int newElement = 25;

    // Shift elements to the right
    for (int i = length - 1; i >= index; i--) {
        arr[i + 1] = arr[i];
    }

    // Insert the new element
    arr[index] = newElement;

    // Update the length of the array
    length++;

    // Print the array after insertion
    for (int i = 0; i < length; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Inserting an element at the beginning of a singly linked list:

1. Start with an empty linked list.
2. To insert an element at the beginning:
 - Create a new node.
 - Set the data of the new node to the value you want to insert.
 - Make the new node point to the current head of the list.
 - Update the head of the list to point to the new node.
3. To print the elements of the linked list:
 - Start from the head of the list.
 - Print the data of each node as you traverse the list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void insertAtBeginning(struct Node** head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = *head;
    *head = newNode;
}

void printList(struct Node* head) {
    struct Node* current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
}
```

```

int main() {
    struct Node* head = NULL;

    // Inserting elements at the beginning of the linked list
    insertAtBeginning(&head, 40);
    insertAtBeginning(&head, 30);
    insertAtBeginning(&head, 20);
    insertAtBeginning(&head, 10);

    // Print the linked list
    printList(head);

    return 0;
}

```

Deleting an Element #revise-1

General steps for deleting an element at a specific index in an array:

- Determine the position of the element you want to delete, such as at the beginning, middle, or end of the array.
- Shift all the elements to the left of the deletion position to overwrite the element being deleted.
- Adjust the length of the array to reflect the removal of the element.

Deleting an element at a specific index in an array:

1. Start with an array containing elements.
2. Specify the index of the element you want to delete.
3. To delete the element:
 - Shift the elements to the left, starting from the index of the element to be deleted.
 - Overwrite each element with the value of the next element in the array.

- Update the length of the array by reducing it by 1.
4. To print the updated array:
- Traverse the array and print the elements.

```
#include <stdio.h>

int main() {
    int arr[10] = {10, 20, 30, 40, 50};
    int length = 5;
    int index = 2; // Index of the element to be deleted

    // Shift elements to the left
    for (int i = index; i < length - 1; i++) {
        arr[i] = arr[i + 1];
    }

    // Update the length of the array
    length--;

    // Print the array after deletion
    for (int i = 0; i < length; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Deleting an element from a singly linked list:

Algorithm:

1. Start with an empty linked list.
2. To insert elements at the beginning:
 - Create a new node with the value you want to insert.
 - Make the new node point to the current head of the list.

- Update the head of the list to point to the new node.
3. To delete an element with a specific value:
- Start from the head of the list and keep track of the previous node as well.
 - Traverse the list until you find the node with the value you want to delete or reach the end of the list.
 - If the node is found:
 - Update the previous node's `next` pointer to skip the node you want to delete.
 - Free the memory of the node you want to delete.
4. To print the elements of the linked list:
- Start from the head of the list.
 - Print the value of each node as you go along.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void deleteNode(struct Node** head, int value) {
    struct Node* current = *head;
    struct Node* previous = NULL;

    // Find the node containing the value
    while (current != NULL && current->data != value) {
        previous = current;
        current = current->next;
    }

    // If the node is found
```

```

    if (current != NULL) {
        // If it's the first node
        if (previous == NULL) {
            *head = current->next;
        }
        // If it's not the first node
        else {
            previous->next = current->next;
        }

        // Free the memory
        free(current);
    }
}

void printList(struct Node* head) {
    struct Node* current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
}

int main() {
    struct Node* head = NULL;

    // Create the linked list
    head = (struct Node*)malloc(sizeof(struct Node));
    head->data = 10;
    head->next = (struct Node*)malloc(sizeof(struct Node));
    head->next->data = 20;
    head->next->next = (struct Node*)malloc(sizeof(struct Node));
    head->next->next->data = 30;
    head->next->next->next = NULL;

    // Delete an element from the linked list

```

```
deleteNode(&head, 20);

// Print the updated linked list
printList(head);

return 0;
}
```

Sorting #revise-1

Sorting is a fundamental operation in data structures and involves arranging a collection of elements in a specific order. The order can be ascending or descending, depending on the sorting algorithm and the desired outcome.

Some common sorting algorithms include:

1. **Bubble Sort:** This algorithm repeatedly compares adjacent elements and swaps them if they are in the wrong order until the entire array is sorted. It has a time complexity of $O(n^2)$ in the average and worst cases.
2. **Selection Sort:** This algorithm divides the array into sorted and unsorted portions. It repeatedly finds the minimum element from the unsorted portion and swaps it with the first unsorted element. Selection sort also has a time complexity of $O(n^2)$.
3. **Insertion Sort:** This algorithm builds a sorted portion of the array by iteratively inserting elements from the unsorted portion into their correct positions. Insertion sort is efficient for small data sets and has a time complexity of $O(n^2)$.
4. **Merge Sort:** It divides the array into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted array. Merge sort has a time complexity of $O(n \log n)$ and is known for its stability.
5. **Heap Sort:** This algorithm uses a binary heap data structure to sort elements. It builds a max heap from the array, repeatedly removes the maximum element from the heap and places it at the end of the sorted portion. Heap sort has a time complexity of $O(n \log n)$.

Bubble Sort #revise-1

Algorithm:

1. Start with an unsorted list of elements.
2. Compare the first element with the second element. If the first element is greater than the second element, swap them.
3. Move to the next pair of adjacent elements and repeat the comparison and swapping process.
4. Continue this process until you reach the end of the list.
5. After the first iteration, the largest element will be in its correct position at the end of the list.
6. Repeat steps 2-5 for the remaining unsorted portion of the list (excluding the already sorted elements).
7. Keep repeating steps 2-6 until the entire list is sorted.

```
#include <stdio.h>

void bubbleSort(int arr[], int size) {
    int i, j;
    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - i - 1; j++) {
            // Compare adjacent elements and swap if necessary
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {5, 2, 8, 12, 3};
```

```
int size = sizeof(arr) / sizeof(arr[0]);

bubbleSort(arr, size);

printf("Sorted array: ");
for (int i = 0; i < size; i++) {
    printf("%d ", arr[i]);
}

return 0;
}
```

Searching #revise-1

Searching in data structures involves finding a specific element or a set of elements within a given collection of data.

Here are some commonly used search algorithms:

1. Linear Search: This is the simplest searching algorithm, where each element in the collection is checked sequentially until a match is found or the entire collection is traversed. Linear search has a time complexity of $O(n)$.
2. Binary Search: Binary search is an efficient searching algorithm used on sorted collections. It repeatedly divides the collection in half and compares the target element with the middle element. Binary search has a time complexity of $O(\log n)$ and requires a sorted collection.
3. Hashing: Hashing is a technique used to store and retrieve data in constant time, typically achieved through the use of a hash table.

Linear Search #revise-1

Linear search is a simple searching algorithm used to find a specific element in a collection of elements.

The algorithm works as follows:

1. Start from the first element of the collection.

2. Compare the current element with the target element you are searching for.
3. If the current element matches the target element, return the index at which it was found.
4. If the current element does not match the target element, move to the next element in the collection.
5. Repeat steps 2 to 4 until a match is found or the end of the collection is reached.

If the target element is found, the algorithm returns its index. If the target element is not present in the collection, the algorithm typically returns a special value, such as -1, to indicate that the element was not found.

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i; // Return the index where the element is found
        }
    }
    return -1; // Return -1 if the element is not found
}

int main() {
    int arr[] = {10, 5, 8, 3, 2};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 8;

    int index = linearSearch(arr, size, target);

    if (index != -1) {
        printf("Element %d found at index %d\n", target, index);
    } else {
        printf("Element %d not found in the array\n", target);
    }
}
```

```
    return 0;
}
```

Binary Search #revise-1

Binary search algorithm:

1. Start with a sorted array of elements.
2. Set two pointers, `low` and `high`, to the start and end of the array, respectively.
3. Calculate the `mid` index as the average of `low` and `high`.
4. Compare the target value with the element at the `mid` index:
 - If the target value is equal to the element at the `mid` index, the search is successful, and the index is returned.
 - If the target value is less than the element at the `mid` index, set `high` to `mid - 1` to search the left half of the array.
 - If the target value is greater than the element at the `mid` index, set `low` to `mid + 1` to search the right half of the array.
5. Repeat steps 3 and 4 until the target value is found or the `low` pointer becomes greater than the `high` pointer.
6. If the target value is not found, return a special value (e.g., -1) to indicate that the target is not present in the array.

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int low, int high, int target) {
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid; // Target found, return the index
        } else if (arr[mid] < target) {
            low = mid + 1; // Target is in the right half of the array
        } else {
            high = mid - 1; // Target is in the left half of the array
        }
    }
    return -1; // Target not found
}
```

```

        } else {
            high = mid - 1; // Target is in the left half of the array
        }
    }

    return -1; // Target not found, return -1
}

int main() {
    int arr[] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 12;

    int index = binarySearch(arr, 0, n - 1, target);

    if (index != -1) {
        printf("Target found at index %d\n", index);
    } else {
        printf("Target not found in the array\n");
    }

    return 0;
}

```

Multi-Dimensional Array

A multi-dimensional array is an array data structure that contains more than one dimension or index for accessing its elements.

Pointer & Pointer Array

A pointer is a variable that stores the memory address of another variable. In other words, it "points" to the location in memory where a value is stored.

```

int main() {
    int x = 10; // Variable x
}

```

```

int* ptr;    // Pointer variable

ptr = &x;    // Assign the memory address of x to ptr

printf("Value of x: %d\n", x);    // Output: 10
printf("Value of x via ptr: %d\n", *ptr);    // Output: 10
printf("Memory address of x: %p\n", &x);    // Output: Some memory address
printf("Value of ptr: %p\n", ptr);    // Output: Same memory address

return 0;
}

```

A pointer array is an array of pointers. Each element of the array is a pointer that can point to a memory location.

```

int main() {
    int arr1[] = {10, 20, 30};
    int arr2[] = {40, 50, 60};
    int* arrPtr[2];    // Pointer array

    arrPtr[0] = arr1;    // Assign the memory address of arr1 to arrPtr[0]
    arrPtr[1] = arr2;    // Assign the memory address of arr2 to arrPtr[1]

    printf("Value at arr1[0]: %d\n", arrPtr[0][0]);    // Output: 10
    printf("Value at arr2[2]: %d\n", arrPtr[1][2]);    // Output: 60

    return 0;
}

```

Records

A record refers to a collection of related data items that are grouped together under a single name.

Sparse Matrix

A sparse matrix is a matrix that contains a significant number of zero elements compared to its total size.

Triangular & Tridiagonal Matrix

Triangular Matrix:

- A triangular matrix is a special type of square matrix where all the elements above or below the main diagonal are zero.
- If the elements above the main diagonal are zero, it is called a lower triangular matrix.
- If the elements below the main diagonal are zero, it is called an upper triangular matrix.
- The main diagonal is the line of elements from the top-left to the bottom-right of the matrix.

```
#include <stdio.h>

#define N 3 // Size of the matrix

int main() {
    int matrix[N][N] = {
        {1, 0, 0},
        {4, 5, 0},
        {7, 8, 9}
    };

    // Accessing and printing elements of the lower triangular matrix
    printf("Lower Triangular Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j <= i; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}
```

```

    // Accessing and printing elements of the upper triangular matrix
    printf("Upper Triangular Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = i; j < N; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Tridiagonal Matrix:

- A tridiagonal matrix is a special type of square matrix where the only non-zero elements are on the main diagonal, the diagonal above the main diagonal (superdiagonal), and the diagonal below the main diagonal (subdiagonal).
- All other elements outside these three diagonals are zero.

```

#include <stdio.h>

#define N 4 // Size of the matrix

int main() {
    int matrix[N][N] = {
        {2, 3, 0, 0},
        {1, 4, 7, 0},
        {0, 5, 8, 11},
        {0, 0, 6, 9}
    };

    // Accessing and printing elements of the tridiagonal matrix
    printf("Tridiagonal Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {

```

```
        if (i == j || i == j + 1 || i == j - 1) {  
            printf("%d ", matrix[i][j]);  
        } else {  
            printf("0 ");  
        }  
    }  
    printf("\n");  
}  
  
return 0;  
}
```